

TME 1 : Rappels Java & Collections

UE: Programmation Concurrente - LU3IN001

Ce TME a pour but de remettre en pratique les notions de base de Java vues en cours et en TD : création de classes, entrées/sorties (I/O), et utilisation des Collections (List, Map).

Exercice 1 : Modélisation d'un Produit

Objectif : Définir la classe représentant les données.

1. Créez une classe 'Produit' dans le package 'tme1'.
2. Ajoutez les attributs privés : id (int), nom (String), prix (double).
3. Créez le constructeur prenant ces 3 paramètres.
4. Générez les accesseurs (getters).
5. Redéfinissez toString() pour afficher : '[ID: 101] Nom - Prix'.

Exercice 2 : Lecture de données (I/O)

Objectif : Lire le fichier 'stock.csv' contenant : 101;Souris;25.50

1. Créez une classe 'GestionStock'.
2. Écrivez une méthode : public static List<Produit> chargerFichier(String chemin).
3. Utilisez BufferedReader et FileReader (bloc try-with-resources).
4. Découpez chaque ligne avec split(';') et convertissez les types (Integer.parseInt, Double.parseDouble).
5. Gérez les exceptions : FileNotFoundException et NumberFormatException.

Exercice 3 : Manipulation de Listes

1. Dans le main, appelez chargerFichier pour récupérer une ArrayList<Produit>.
2. Affichez le nombre d'articles chargés via size().
3. Parcourez et affichez la liste avec une boucle 'foreach'.

Exercice 4 : Dictionnaires (Map)

Objectif : Accès rapide par identifiant.

1. Convertissez la liste en une HashMap<Integer, Produit> (Clé = ID).
 2. Recherchez un produit spécifique (ex: ID 103) via map.get().
- Rappel : L'accès dans une HashMap est en O(1) contre O(n) pour une liste.

Exercice 5 : Tri et Comparateurs

1. Triez la liste des produits du moins cher au plus cher.
2. Utilisez Collections.sort() avec une expression lambda pour le comparateur :
(p1, p2) -> Double.compare(p1.getPrix(), p2.getPrix())